

Flurry Advertising

Android SDK Instructions

SDK version 3.0.5
Updated: 10/26/2012

Welcome to Flurry Advertising!

This README contains:

1. Introduction
2. Basic Integration
3. Additional Ad Controls (Optional)
4. Enabling Ad Network Mediation (Optional)
5. Request Configuration (Optional)
6. Implementing Ad Listener (Optional)

---

1. Introduction

Flurry Advertising is a flexible ad serving solution that allows you to easily manage the complete monetization of your mobile applications. Through integration with Flurry Analytics and various mobile ad networks, publishers can easily generate the maximum value from ad campaigns. Both AppCircle and AppSpot (direct and custom network) ad campaigns are created and managed in the same interface.

With AdUnity you will be able to:

1. Define your inventory
2. Traffic your ad campaigns
3. Track & optimize your performance

The Flurry SDK contains all the existing Flurry Analytics functionality as well as the new Advertising functionality. It is designed to be as easy as possible with a basic setup completed in under 5 minutes.

These instructions assume that you have already integrated Flurry Analytics into your application. If you have not done so, please refer to **Analytics-README** to get started. AdUnity is contained in the same JAR file that contains the Analytics libraries so no additional libraries are required.

---

2. Basic Integration

To integrate Flurry Advertising into your Android application, just complete two steps:

Step 1. Initialize ad system

Declare the Activity “FlurryFullscreenTakeoverActivity” in your AndroidManifest.xml file:

```
<activity android:name="com.flurry.android.FlurryFullscreenTakeoverActivity"
android:configChanges="keyboard|keyboardHidden|orientation|screenLayout|
uiMode" ></activity>
```

Add the following line to your app after the call to FlurryAgent.onStartSession() in your Main activity. This only needs to be called once. Note: please see **Analytics-README** for details on FlurryAgent.onStartSession():

```
FlurryAgent.initializeAds(Context context);
```

The parameters are as follows:

- **Context context** - Context you want the ad to reside within. We advise using the current Activity.

Step 2. Requesting an ad

Call getAd after the session has started. Note: please see **Analytics-README** for details on FlurryAgent.onStartSession():

```
FlurryAgent.getAd(Context context, String adSpace, ViewGroup viewSpace,
FlurryAdSize size, long timeout);
```

If the ad request is filled then this will return true, and false if no ad was available. The parameters are as follows:

- **Context context** - Context you want the ad to reside within. We advise using the current Activity.
- **String adSpace** - Name of the adSpace that you defined on the Flurry website
- **ViewGroup viewSpace** - ViewGroup inside the Context that you want the ad to reside in
- **FlurryAdSize size** - The default size for your adSpace. This is overwritten by any value set on the server.
- **long timeout** - Maximum time (in milliseconds) this method should block before it returns

Note that the same ad call is used for all ad formats and sizes including Banner, Takeover and Custom ad formats. In order to specify the format and size, you should log into the Flurry Website and select the Publisher tab to specify the configuration for the adSpace name you have chosen.

---

3. Additional Ad Controls (Optional)

Checking if an ad is available

Flurry provides a method to allow your app to check for the availability of an ad without displaying it. This is useful if you provide an option for a user to view an ad (e.g. - display a button to View a Video). Simply add the following call within your app with the name of the adSpace that will ultimately display the ad

```
FlurryAgent.isAdAvailable(Context context, String adSpace, FlurryAdSize size,
long timeout)
```

This call will check the Flurry server for the availability of an ad. If an ad is available then this will return true, and false if no ad was available. If this returns true you are guaranteed an ad will be available for the given adSpace when FlurryAgent.getAd is called. The parameters are as follows:

- **Context context** - Context you want the ad to reside within. We advise using the current Activity.
- **String adSpace** - Name of the adSpace that you defined on the Flurry website
- **FlurryAdSize size** - The default size for your adSpace. This is overwritten by any value set on the server.
- **long timeout** - Maximum time (in milliseconds) this method should block before it returns.

Removing an ad

Flurry manages the lifecycle of the ads it displays, however, you can exercise finer control over display by choosing when to add and remove the ads from your app. To remove an ad just call

```
FlurryAgent.removeAd(Context context, String myAdSpaceName, ViewGroup
viewSpace);
```

The parameters are as follows:

- **Context context** - Context you want the ad to reside within. We advise using the current Activity.
- **String myAdSpaceName** - Name of the adSpace that you defined on the Flurry website
- **ViewGroup viewSpace** - ViewGroup inside the Context that you want the ad to reside in

---

4. Enabling Ad Network Mediation (Optional)

Once you have your Ad Spaces set up, you will have the option of selecting 3rd party ad networks to serve ads into your Ad Spaces using the Flurry website (in addition to AppCircle, AppSpot and your own ads). You can change which ad networks serve ads at any time on the Flurry website, but in order to enable them you need to add the ad network SDKs into your application and configure them. The list of currently supported Ad Networks contains:

- Admob - SDK Version 4.3.1
- Millennial - SDK Version 4.5.1
- InMobi - SDK Version 300
- Mobclix - SDK Version 3.2.0
- JumpTap - SDK Version 1.1.10.4

To implement an Ad Network you must perform the following steps:

1. Include the Ad Network Android SDK with your app and add it to the build path. Follow the instructions from the Ad network on how to complete this step.
2. Create the proper “activity” and “meta-data” tags in AndroidManifest.xml
    a. the first meta-data tag instructs the SDK about how to find the API_KEY
    b. the second meta-data tag instructs the SDK whether to request test ads
3. Add your API_KEY in strings.xml

Here is the example of implementing the Admob SDK into AppSpot:

AndroidManifest.xml

```
<activity android:name="com.google.ads.AdActivity"
      android:configChanges="keyboard|keyboardHidden|orientation|
      screenLayout|uiMode|screenSize|smallestScreenSize"/>

<meta-data android:name="com.flurry.admob.MY_AD_UNIT_ID"
      android:value="@string/appSpot_admob_apikey"/>
<meta-data android:name="com.flurry.admob.test"
      android:value="true"/>
```

strings.xml

```
<string name="appSpot_admob_apikey">YOUR_API_KEY</string>
```

---

5. Request Configuration (Optional)

There are a number of configuration parameters that you can use to modify the behavior of your ad spaces.

Option 1. Set Location

Add a call to set the location (lat,long) that you want associated with the ad request, to be used with geographical targeting:

```
FlurryAgent.setLocation(latitude, longitude);
```

Option 2. User Cookies

Add a call to identify any user specific information you want associated with the ad request:

```
FlurryAgent.addUserCookie("test_key", "test_value");
```

To remove any user cookies call:

```
FlurryAgent.clearUserCookies();
```

Option 3. Keyword Targeting

Add a call to specify keywords to be used when targeting ads:

```
FlurryAgent.setTargetingKeywords(Map<String,String>);
```

---

6. Implementing Ad Listener (Optional)

To be notified of certain events during the full lifecycle of the Ad, you can implement the com.flurry.IListener interface and then call the setAdListener method to attach the com.flurry.IListener to the FlurryAgent. When you implement the com.flurry.IListener interface you will implement the following callback methods:

- shouldDisplayAd(String myAdSpaceName, FlurryAdType type)

    - This method will be called if there is an ad available to display for that adSpace. The FlurryAdType is the following enumeration: `WEB_BANNER, WEB_TAKEOVER, VIDEO_TAKEOVER`. You can choose whether or not to show the available ad by returning either true or false. If shouldDisplayAd returns false, then the ad will not be displayed and getAd() will return false. If shouldDisplayAd returns true, then the ad will be displayed and getAd() will return true.
- willDismissAd(String myAdSpaceName)
    - This method will be called when the user dismisses the current Ad for the provided Ad Space name.
- willLeaveApplication(String myAdSpaceName)
    - This method will be called when the user is leaving the application after following events associated with the current Ad in the provided Ad Space name.
- didReward(String adSpace, Map<String, String> rewards)
    - This method will be called if the ad resulted in rewards to be issued to the user following events associated with the current Ad in the provided Ad Space name.

Example usage:

```
public class MyAdListener implements IListener {

	@Override
	public boolean shouldDisplayAd(String myAdSpaceName, FlurryAdType type)
	{
		// Decide if the Ad should be displayed
		return true;
	}

	@Override
	public void willDismissAd(String myAdSpaceName)
	{
		// Handle the user dismissing the ad
	}

	@Override
	public void willLeaveApplication(String myAdSpaceName)
	{
		// Handle the user leaving the application

	}
	@Override
	public void didReward(String myAdSpaceName, Map<String, String> arg1)
	{
		// Handle rewards earned by the user
	}
}

IListener myAdListener = new MyAdListener(); // Create an instance
FlurryAgent.setAdListener(myAdListener);     // Register the listener
```

---

7. Using ProGuard (Optional)

If you plan to run ProGuard on your APK before releasing your app, you will need to add the following to your “proguard.cfg” file:

```
-keep class com.flurry.** { *; }
-dontwarn com.flurry.**
```